

Guia para explicar el codigo en sustentacion

Generador de Recetas con Inventario | FastAPI + PostgreSQL + OpenRouter + Docker

Integrantes: Sean Paul Marquez Toro, Reyner David Barbosa de la Rosa y Ruben Andres Corro Blanco

Respuesta de apertura

El proyecto es una aplicacion web donde cada usuario registra los ingredientes que tiene en casa. Con ese inventario, la app recomienda recetas por coincidencia de productos de canasta familiar y tambien puede generar una receta completa con un LLM usando OpenRouter.

1. Mapa rapido del proyecto

Ruta	Funcion	Que decir si la abre el profesor
app/main.py	Punto de entrada FastAPI	Aqui se crea la app, se monta /static, se inicializa la base de datos y se registran los routers.
app/config.py	Variables de entorno	Lee DATABASE_URL, SECRET_KEY, OPENROUTER_API_KEY, OPENROUTER_MODEL y LLM_DRY_RUN sin quemar secretos en codigo.
app/database.py	Conexion SQLAlchemy	Crea el engine, maneja sesiones por peticion y crea tablas al iniciar.
app/models/	Tablas de negocio	Define usuarios, ingredientes, recetas y calificaciones con relaciones entre ellas.
app/schemas/	Validacion Pydantic	Controla datos de entrada y salida: correos, cantidades, porciones, ratings y estructura JSON de recetas.
app/routers/	Endpoints REST	Agrupaa autenticacion, ingredientes, recetas y pagina principal.

Ruta	Funcion	Que decir si la abre el profesor
app/services/	Logica interna	Separa seguridad, recomendaciones locales y generacion con OpenRouter.
templates/ + static/	Frontend	HTML, CSS y JS consumen la API con fetch y renderizan la experiencia visual.
docker-compose.yml	Produccion	Levanta PostgreSQL, FastAPI y Caddy para dominio y SSL.
deploy/Caddyfile	Proxy y HTTPS	Caddy recibe el dominio, emite SSL y envia el trafico al contenedor app:8000.

2. Como arranca la aplicacion

El punto de entrada es app/main.py. Alli se instancia FastAPI con titulo, descripcion y version. Tambien se define un lifespan que ejecuta init_db() al iniciar, por eso la aplicacion prepara las tablas antes de atender peticiones.

- app.mount('/static', ...) publica CSS, JS, imagenes y favicon.
- include_router(pages.router) sirve la interfaz web en /.
- include_router(auth.router) registra endpoints de registro, login y /me.
- include_router(ingredients.router) registra el CRUD del inventario.
- include_router(recipes.router) registra recomendaciones, generacion, favoritos y ratings.

Frase lista

La app arranca en app/main.py. FastAPI conecta los routers, monta archivos estaticos y ejecuta init_db() para preparar la base de datos al inicio.

3. Configuracion y variables de entorno

La configuracion esta centralizada en app/config.py mediante una clase Settings. Esto permite cambiar base de datos, modelo LLM, dominio o llaves sin modificar el codigo fuente.

- DATABASE_URL: cadena de conexion a PostgreSQL en produccion o SQLite local.
- SECRET_KEY: clave usada para firmar tokens JWT.

- OPENROUTER_API_KEY: token del proveedor LLM.
- OPENROUTER_MODEL: modelo usado; por defecto openai/gpt-4o-mini.
- LLM_DRY_RUN: modo de prueba para generar fallback sin llamar al proveedor externo.

Frase lista

No dejamos secretos quemados. El código lee las variables desde el entorno, y Docker Compose las inyecta al contenedor.

4. Base de datos y modelos

La base se maneja con SQLAlchemy. En app/database.py se crea el engine y SessionLocal. Cada endpoint recibe una sesión mediante Depends(get_db), lo que abre la conexión, ejecuta la operación y la cierra al final.

Modelo	Tabla	Explicación
User	users	Guarda email, nombre, password_hash y relaciones con inventario, recetas y ratings.
Ingredient	ingredients	Guarda cada ingrediente con owner_id, nombre, cantidad, unidad, notas y fechas.
Recipe	recipes	Guarda recetas generadas: nombre, descripción, ingredientes JSON, pasos, faltantes, tips, nutrición, tags y prompt.
Rating	ratings	Guarda calificación de 1 a 5 y comentario; tiene restricción única por usuario y receta.

Frase lista

Cada ingrediente y receta pertenece a un usuario por owner_id. Por eso las consultas filtran por el usuario autenticado y no mezclan datos.

5. Validacion con Pydantic

Los schemas en `app/schemas/` validan datos antes de guardarlos o devolverlos. Esto evita entradas invalidas y tambien define la forma oficial de las respuestas de la API.

- `UserCreate` valida correo y exige contrasena minima de 8 caracteres.
- `IngredientCreate` exige nombre no vacio, cantidad mayor que cero y unidad valida.
- `RecipeGenerateRequest` limita porciones entre 1 y 12 y tiempo entre 5 y 240 minutos.
- `GeneratedRecipe` define el JSON que debe entregar el LLM: `nombre_plato`, `ingredientes`, `pasos`, `nutricion`, `etiquetas` y `dificultad`.
- `RatingCreate` restringe la calificacion entre 1 y 5.

Frase lista

Pydantic es la primera defensa de calidad: valida entradas del usuario y valida tambien la estructura que viene del LLM.

6. Autenticacion y seguridad

El registro y login estan en `app/routers/auth.py`. La seguridad esta en `app/services/security.py`. La contrasena no se guarda en texto plano: se guarda como hash PBKDF2-SHA256 con salt. Cuando el usuario inicia sesion, la app genera un token JWT firmado.

- El usuario se registra en `/api/auth/register`.
- La contrasena pasa por `hash_password()` y se guarda como `password_hash`.
- En login se compara con `verify_password()`.
- Si las credenciales son correctas, `create_access_token()` genera un Bearer token.
- `get_current_user()` lee el token, lo valida y retorna el usuario autenticado.

Frase lista

Las rutas protegidas usan `Depends(get_current_user)`. Asi el backend sabe quien esta haciendo la peticion y puede filtrar sus datos.

7. Endpoints de ingredientes

El router `app/routers/ingredients.py` implementa el inventario personal. Permite listar, crear, editar y eliminar ingredientes. Tambien tiene `/summary` para calcular metricas del inventario.

- `GET /api/ingredients`: lista solo ingredientes del usuario actual.

- POST /api/ingredients: crea un ingrediente asociado al current_user.id.
- PUT /api/ingredients/{id}: actualiza si el ingrediente pertenece al usuario.
- DELETE /api/ingredients/{id}: elimina si pertenece al usuario.
- GET /api/ingredients/summary: calcula total de items, unidades, score y sugerencias.

8. Recomendaciones locales

Las recomendaciones estan en app/services/recommendation_service.py. No dependen del LLM; usan plantillas locales de recetas con productos de canasta familiar. Esto hace que respondan rapido y siempre funcionen.

- Se normalizan los nombres de ingredientes del inventario.
- Se comparan contra alias: huevo/huevos, platano/platano, atun/atún, etc.
- Cada plantilla define ingredientes requeridos y opcionales.
- Se calcula match_score con hasta 80 puntos por requeridos y 20 por opcionales.
- Se construye un breakdown: disponible, faltante, opcional y basico.
- Se ordenan las mejores opciones y se devuelven hasta 18 recomendaciones.

Frase lista

Las recomendaciones son un motor local de reglas y plantillas. El LLM se usa despues, cuando el usuario quiere generar una receta completa.

9. Generacion con OpenRouter

La generacion esta en app/services/llm_service.py y el endpoint que la llama esta en app/routers/recipes.py. El backend no manda una conversacion abierta; construye un prompt controlado y exige una respuesta JSON.

- El usuario presiona Generar receta IA.
- El frontend envia opciones: porciones, tipo de comida, objetivo, tiempo, restricciones e instrucciones extra.
- El backend consulta el inventario del usuario.
- build_recipe_prompt() arma el prompt con inventario y contrato JSON.
- generate_recipe_from_inventory() llama a OpenRouter en /chat/completions.
- parse_recipe_response() extrae y valida el JSON con GeneratedRecipe.
- La receta se guarda en la tabla recipes con prompt y raw_response para trazabilidad.

Frase lista

El LLM no decide la estructura de la aplicación. Solo genera el contenido de la receta, y el backend valida que venga en el formato esperado.

10. Historial, favoritos y calificaciones

El router de recetas también permite consultar historial, buscar por texto, filtrar favoritas, marcar favoritas, calificar y eliminar recetas.

- GET /api/recipes devuelve recetas del usuario con su rating si existe.
- PATCH /api/recipes/{id}/favorite actualiza is_favorite.
- POST /api/recipes/{id}/ratings crea o actualiza la calificación del usuario.
- DELETE /api/recipes/{id} elimina la receta si pertenece al usuario.

11. Frontend: HTML, CSS y JavaScript

La interfaz está en templates/index.html y static/app.js. El frontend guarda el token en localStorage y usa fetch para consumir la API REST.

- state guarda token, usuario, ingredientes, recetas, recomendaciones y resumen.
- api() agrega el header Authorization: Bearer cuando hay sesión.
- loadAll() carga ingredientes, recetas, resumen y recomendaciones en paralelo.
- renderIngredients(), renderRecommendations() y renderRecipes() pintan la pantalla.
- Los chips de canasta familiar agregan productos comunes rápidamente.
- El botón Generar receta IA envía POST /api/recipes/generate.

Frase lista

El frontend es una interfaz que consume una API real. No trabaja con datos falsos: todo sale de los endpoints de FastAPI.

12. Docker, dominio y SSL

El proyecto se ejecuta en producción con Docker Compose. Hay tres servicios principales: db, app y caddy.

Servicio	Tecnologia	Funcion
db	postgres:16-alpine	Base de datos PostgreSQL con volumen persistente.
app	FastAPI + Uvicorn	Backend Python que expone la API y la interfaz.
caddy	Caddy 2	Proxy inverso, HTTPS automatico y redireccion hacia app:8000.

Frase lista

La arquitectura en produccion es: navegador -> dominio con SSL en Caddy -> contenedor FastAPI -> PostgreSQL. OpenRouter queda como proveedor externo.

13. Flujo completo de una receta

- Usuario inicia sesion y recibe token JWT.
- Agrega ingredientes al inventario.
- La app muestra resumen y recomendaciones locales.
- Usuario configura preferencias y genera receta.
- FastAPI consulta inventario, arma prompt y llama OpenRouter.
- La respuesta JSON se valida con Pydantic.
- La receta se guarda en PostgreSQL.
- El frontend actualiza historial, permite marcar favorita, calificar y copiar.

14. Preguntas tipicas del profesor

Pregunta	Respuesta corta
Que LLM usan?	OpenRouter como proveedor, con modelo configurable por OPENROUTER_MODEL; por defecto openai/gpt-4o-mini.
Donde esta el prompt?	En build_recipe_prompt() dentro de app/services/llm_service.py.
Como protegen datos?	Con JWT, hash de contraseñas y filtros por owner_id en ingredientes y recetas.

Pregunta	Respuesta corta
Que base de datos usan?	PostgreSQL en produccion mediante Docker Compose; SQLite solo como fallback local si no hay DATABASE_URL.
Las recomendaciones son IA?	Son un motor local por reglas y plantillas; la IA se usa para generar la receta completa.
Como validan la respuesta del LLM?	Se exige JSON, se parsea y se valida con el schema GeneratedRecipe de Pydantic.
Como se despliega?	Docker Compose levanta db, app y caddy. Caddy gestiona dominio, proxy inverso y SSL.
Como prueban el proyecto?	Hay tests en tests/ para API, validaciones, recomendaciones y servicio LLM. El ultimo resultado local fue 14 passed.

15. Mini guion para decirlo de corrido

El proyecto es un generador de recetas con inventario. Primero el usuario se registra e inicia sesion. Luego agrega ingredientes, que se guardan en PostgreSQL asociados a su usuario. Con ese inventario, la app calcula recomendaciones locales usando plantillas de canasta familiar y un puntaje de coincidencia. Si el usuario quiere una receta completa, FastAPI arma un prompt controlado y consulta OpenRouter. La respuesta debe venir en JSON, se valida con Pydantic y se guarda como historial. El frontend consume los endpoints REST con fetch, muestra recetas, favoritos, calificaciones y desglose de ingredientes. En produccion todo corre con Docker Compose: PostgreSQL, FastAPI y Caddy, que maneja el dominio y SSL.

16. Checklist antes de abrir Visual Studio

- Abrir app/main.py para mostrar punto de entrada.
- Abrir app/routers/recipes.py para mostrar generacion y guardado.
- Abrir app/services/llm_service.py para mostrar prompt y OpenRouter.
- Abrir app/services/recommendation_service.py para mostrar recomendaciones de canasta familiar.
- Abrir app/models/ para mostrar entidades de base de datos.
- Abrir docker-compose.yml para mostrar PostgreSQL, app y Caddy.
- Abrir /docs en produccion para mostrar Swagger.